

Ex.No: 4.A	Secure Encryption and Decryption of Hospital Patient Records Using the DES Algorithm
Date:	

AIM:

ALGORITHM:

CODE:

```
/pip install pycryptodome  
from Crypto.Cipher import DES  
from Crypto.Util.Padding import pad, unpad  
key = b'12345678'
```

```
cipher = DES.new(key, DES.MODE_ECB)

plaintext = input("Enter Plaintext: ")

ciphertext = cipher.encrypt(pad(plaintext.encode(), DES.block_size))

decipher = DES.new(key, DES.MODE_ECB)

print("Encrypted Data:", ciphertext.hex())

decrypted = unpad(decipher.decrypt(ciphertext), DES.block_size)

print("Decrypted Data:", decrypted.decode())
```

OUTPUT:

```
Requirement already satisfied: pycryptodome in /usr/local/lib/python3.12/dist-packages (3.23.0)
Enter Plaintext: Hello
Encrypted Data: a1599fcd6c611fcf
Decrypted Data: Hello
```

RESULT:

Ex.No: 4.B	ImplemSecure Online Banking Transaction Using the RSA Public-Key Cryptographic Algorithm
Date:	

AIM:

ALGORITHM:

CODE:

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
```

return None

231401116

```
p = 17  
q = 11
```

```
n = p * q  
phi = (p - 1) * (q - 1)
```

```
e = 7
```

```
if gcd(e, phi) != 1:  
    print("Choose another value for e.")  
else:  
    d = mod_inverse(e, phi)
```

```
print("Public Key:", (e, n))  
print("Private Key:", (d, n))
```

```
message = int(input("Enter Message (<187): "))
```

```
if message >= n:  
    print("Message must be less than", n)  
else:  
    cipher = pow(message, e, n)  
    print("Encrypted Message:", cipher)  
  
    plain = pow(cipher, d, n)  
    print("Decrypted Message:", plain)
```

Enter Keyword : PLAY
Enter Plaintext : HELLO

Key Matrix

P L A Y B
C D E F G
H I K M N
O Q R S T
U V W X Z

Ciphertext : KCYVPQ
Decrypted Text : HELLO

RESULT:

OUTPUT:	Software File Integrity Verification Using the MD5 Hashing Algorithm
Date:	

AIM:

ALGORITHM:

CODE:

```
import hashlib

message = input("Enter Message: ")

hash_value = hashlib.md5(message.encode())

print("MD5 Digest:", hash_value.hexdigest())
```

Enter Message: Cryptology

MD5 Digest: c6ae9d0a089311bab119969161ca5cce

RESULT:

OUTPUT:	Scenario Cloud-Based Document Integrity Verification Using the SHA-1 Hashing Algorithm
Date:	

AIM:

ALGORITHM:

CODE:

```
import hashlib
message=input("Enter Message: ")
hash_value=hashlib.sha1(message.encode())
print("SHA-1 Digest:",hash_value.hexdigest())
```

Enter Message: Cryptology

SHA-1 Digest: ba7160026c971c61eca61f590d68b3de9fadc0f3

RESULT: